Lecture 5. Import and Tidy Data

Tibbles, readr parsing, tidy data, pivoting, separating and uniting

Contents

5.1 Tibbles **2**

5.2 Data import with readr **3**

5.2.1 Parsing a vector 3

5.2.2 Parsing a file 5

5.2.3 Writing files and other formats 5

5.3 Tidy data **6**

5.4 Gathering and spreading **7**

5.5 Separating and uniting **8**

5.1 Tibbles

Motivation. Every tidyverse function takes and returns a tibble. Tibbles are data frames with a few old behaviours removed, and knowing those differences avoids surprises when moving between base R and the tidyverse.

A **tibble** is a data frame that does less: it never changes the type of inputs (strings are not turned into factors), never changes variable names, never creates row names, and complains when a variable does not exist.

```
as_tibble(iris)                          # data frame -> tibble
```

```
## # A tibble: 150 x 5
##   Sepal.Length Sepal.Width Petal.Length Petal.Width Species
##          <dbl>       <dbl>        <dbl>       <dbl> <fct>
## 1          5.1         3.5          1.4         0.2 setosa
## 2          4.9         3            1.4         0.2 setosa
## 3          4.7         3.2          1.3         0.2 setosa
## 4          4.6         3.1          1.5         0.2 setosa
## 5          5           3.6          1.4         0.2 setosa
## # i 145 more rows
```

```
tibble(x = 1:5, y = 1, z = x ^ 2 + y)  # from vectors; y is recycled
```

```
## # A tibble: 5 x 3
##       x     y     z
##   <int> <dbl> <dbl>
## 1     1     1     2
## 2     2     1     5
## 3     3     1    10
## 4     4     1    17
## 5     5     1    26
```

```
tribble(                                 # "transposed tibble": written row by row
  ~x, ~y, ~z,
  "a", 2, 3.6,
  "b", 1, 8.5
)
```

```
## # A tibble: 2 x 3
##   x         y     z
##   <chr> <dbl> <dbl>
## 1 a         2   3.6
## 2 b         1   8.5
```

`as.data.frame()` converts back, for old functions that do not accept tibbles. Column names may be non-syntactic (e.g. `‘:)‘`, `‘2000‘`) if surrounded by backticks.

Printing. A tibble prints the first 10 rows and the columns that fit on screen, with each column's type. `print(n = 10, width = Inf)` shows more; the defaults are changed by `options(tibble.print_max = n, tibble.print_min = m)` (if there are more than `n` rows, print `m`) and `options(tibble.width = Inf)`.

Subsetting. `$` extracts a column by name, `[[ ]]` by name or position. Unlike data frames, tibbles never do partial matching of names and warn if a column does not exist. In a pipe, the placeholder `.` stands for the incoming object:

```
df <- tibble(x = runif(3), y = rnorm(3))
df$x
```

```
## [1] 0.08075014 0.83433304 0.60076089
```

```
df[[2]]
```

```
## [1] -1.00599730 -0.08433889 -0.55406480
```

```
df %>% .$x
```

```
## [1] 0.08075014 0.83433304 0.60076089
```

5.2 Data import with readr

Motivation. Analysing our own data starts with reading a file. A text file only contains characters; turning them into numbers, dates and factors is guesswork that occasionally goes wrong, so we need to see how readr guesses and how to correct it.

`readr` turns flat files into tibbles:

- `read_csv()` (comma-delimited), `read_csv2()` (semicolon-separated, where the comma is the decimal mark), `read_tsv()` (tab-delimited), `read_delim()` (any delimiter);
- `read_fwf()` (fixed-width), `read_table()` (columns separated by white space), `read_log()` (Apache-style logs).

They share their arguments, so learning `read_csv()` covers them all. The first argument is a path, e.g. `read_csv("data/heights.csv")`; an inline string is convenient for experiments:

```
read_csv("a,b,c
1,2,3
4,5,6")
```

```
## # A tibble: 2 x 3
##       a     b     c
##   <dbl> <dbl> <dbl>
## 1     1     2     3
## 2     4     5     6
```

Useful arguments:

```
read_csv("The first line of metadata
  The second line of metadata
  x,y,z
  1,2,3", skip = 2)                                  # skip n lines
read_csv("# A comment I want to skip
  x,y,z
  1,2,3", comment = "#")                             # drop comment lines
read_csv("1,2,3\n4,5,6", col_names = FALSE)          # no header: X1, X2, X3
read_csv("1,2,3\n4,5,6", col_names = c("x", "y", "z"))
read_csv("a,b,c\n1,2,.", na = ".")                   # symbol for missing values
```

Compared with base R's `read.csv()`, readr is faster, produces tibbles, does not convert strings to factors, and behaves the same on every operating system.

5.2.1 Parsing a vector

`parse_*()` functions turn a character vector into a more specialised vector; they are the building blocks of `read_csv()`.

```
str(parse_logical(c("TRUE", "FALSE", "NA")))
```

```
##  logi [1:3] TRUE FALSE NA
```

```
str(parse_date(c("2010-01-01", "1979-10-14")))
```

```
##  Date[1:2], format: "2010-01-01" "1979-10-14"
```

```
parse_integer(c("1", "231", ".", "456"), na = ".")
```

```
## [1]   1 231  NA 456
```

```
x <- parse_integer(c("123", "345", "abc", "123.45"))
```

```
## Warning: 2 parsing failures.
## row col               expected actual
##   3  -- no trailing characters abc
##   4  -- no trailing characters 123.45
```

```
x
```

```
## [1] 123 345  NA  NA
## attr(,"problems")
## # A tibble: 2 x 4
##     row   col expected               actual
##   <int> <int> <chr>                  <chr>
## 1     3    NA no trailing characters abc
## 2     4    NA no trailing characters 123.45
```

```
problems(x)
```

```
## # A tibble: 2 x 4
##     row   col expected               actual
##   <int> <int> <chr>                  <chr>
## 1     3    NA no trailing characters abc
## 2     4    NA no trailing characters 123.45
```

Failures become `NA` with a warning; `problems()` lists them.

Numbers. Three complications: the decimal mark differs between countries (`.` or `,`); numbers come with context (`"$1000"`, `"10%"`); and grouping marks vary (`1,000,000`, `1.000.000`, `1'000'000`).

```
parse_double("1,23", locale = locale(decimal_mark = ","))
```

```
## [1] 1.23
```

```
parse_number("It cost $123.45")
```

```
## [1] 123.45
```

```
parse_number("123.456.789", locale = locale(grouping_mark = "."))
```

```
## [1] 123456789
```

`parse_number()` ignores non-numeric characters before and after the number.

Strings. A string is stored as bytes; `charToRaw("Hadley")` shows them in hexadecimal. The mapping from bytes to characters is the **encoding**. Modern data are almost always UTF-8, but older systems produce other encodings, e.g. Latin-1 for Western European text or EUC-KR for Korean, which print as garbage unless declared:

```
x2 <- "\xbe\xc8\xb3\xe7\xc7\xcf\xbc\xbc\xbf\xe4"     # bytes of a Korean greeting in EUC
    -KR
parse_character(x2, locale = locale(encoding = "EUC-KR"))
guess_encoding(charToRaw(x2))                        # when the encoding is unknown
```

Factors represent categorical variables with a known set of values; `parse_factor(x, levels = fruit)` warns about values not in `levels`.

Dates and times. Computers store a *date* as the number of days since 1970-01-01, a *date-time* as seconds since midnight 1970-01-01 (UTC), and a *time* as seconds since midnight.

- `parse_datetime()` expects ISO 8601, which orders components from biggest to smallest (year, month, day, hour, minute, second); a missing time is set to midnight.
- `parse_date()` expects year, `-` or `/`, month, `-` or `/`, day.

- `parse_time()` expects hour, `:`, minutes, optionally `:` seconds and am/pm.

```
parse_datetime("2010-10-01T2010")
## [1] "2010-10-01 20:10:00 UTC"

parse_date("2010-10-01")
## [1] "2010-10-01"

parse_time("01:10 am")
## 01:10:00

parse_date("01/02/15", "%d/%m/%y")
## [1] "2015-02-01"
```

Other formats are described with codes such as `%Y` (4-digit year), `%y` (2-digit), `%m`, `%b` (abbreviated month name), `%d`, `%H`, `%M`, `%S`.

5.2.2 Parsing a file

readr guesses each column's type from the first 1000 rows, trying in order: logical (only `T`, `F`, `TRUE`, `FALSE`), integer, double, number (with grouping marks), time, date, date-time; if nothing fits, the column stays character. This fails when the first 1000 rows are a special case (e.g. all missing) or when a column has many missing values. readr's example file `challenge.csv` shows both problems:

```
challenge <- read_csv(readr_example("challenge.csv"))
problems(challenge)
## # A tibble: 0 x 5
## # i 5 variables: row <int>, col <int>, expected <chr>, actual <chr>, file <chr>
```

The column `y` has only missing values in the first 1000 rows and was guessed to be logical. Fix the column types one at a time with an explicit specification:

```
challenge <- read_csv(readr_example("challenge.csv"),
  col_types = cols(x = col_double(), y = col_date()))
tail(challenge)
## # A tibble: 6 x 2
##       x y
##   <dbl> <date>
## 1 0.805 2019-11-21
## 2 0.164 2018-03-29
## 3 0.472 2014-08-04
## 4 0.718 2015-08-16
## 5 0.270 2020-02-04
## # i 1 more row
```

Supplying `col_types` is good practice even when the guess is right: it documents the data and makes the script fail loudly if the data change. Other strategies: `guess_max = 1001` looks at more rows; reading every column as character (`cols(.default = col_character())`) and then calling `type_convert()` applies the guessing heuristic afterwards.

5.2.3 Writing files and other formats

`write_csv()` and `write_tsv()` write UTF-8 files with dates in ISO 8601, but the column types are lost: reading back re-guesses them. `write_rds()` and `read_rds()` store R's custom binary format, which preserves types. Other formats have their own packages: `haven` (SPSS, Stata, SAS), `readxl` (`.xls`, `.xlsx`), `DBI` with a backend such as `RSQLite` (SQL databases), `jsonlite` (JSON) and `xml2` (XML).

5.3 Tidy data

Motivation. The same information can be laid out in a table in many ways, but only one layout lets the tidyverse work without fighting the data. Recognising that layout, and converting other layouts into it, is the core skill of data wrangling.

The tidyr examples `table1`–`table5` contain the same numbers (tuberculosis cases and population of three countries in 1999 and 2000) in different layouts:

```
table1
```

```
## # A tibble: 6 x 4
##   country      year  cases population
##   <chr>       <dbl>  <dbl>      <dbl>
## 1 Afghanistan  1999    745   19987071
## 2 Afghanistan  2000   2666   20595360
## 3 Brazil       1999  37737  172006362
## 4 Brazil       2000  80488  174504898
## 5 China        1999 212258 1272915272
## # i 1 more row
```

```
table2
```

```
## # A tibble: 12 x 4
##   country      year type           count
##   <chr>       <dbl> <chr>          <dbl>
## 1 Afghanistan  1999 cases            745
## 2 Afghanistan  1999 population 19987071
## 3 Afghanistan  2000 cases           2666
## 4 Afghanistan  2000 population 20595360
## 5 Brazil       1999 cases          37737
## # i 7 more rows
```

```
table3
```

```
## # A tibble: 6 x 3
##   country      year rate
##   <chr>       <dbl> <chr>
## 1 Afghanistan  1999 745/19987071
## 2 Afghanistan  2000 2666/20595360
## 3 Brazil       1999 37737/172006362
## 4 Brazil       2000 80488/174504898
## 5 China        1999 212258/1272915272
## # i 1 more row
```

Definition (Tidy data)

A data set is **tidy** if

1. each variable has its own column;
2. each observation has its own row;
3. each value has its own cell.

Only `table1` is tidy. `table2` spreads one observation over two rows; `table3` puts two variables in one cell; `table4a` (cases) and `table4b` (population) use the values of `year` as column names.

Why tidy? A consistent layout means one set of tools works everywhere, and storing variables in columns fits R's vectorised nature: `mutate()` and summary functions operate on columns.

```
table1 %>% mutate(rate = cases / population * 10000)   # rate per 10,000
```

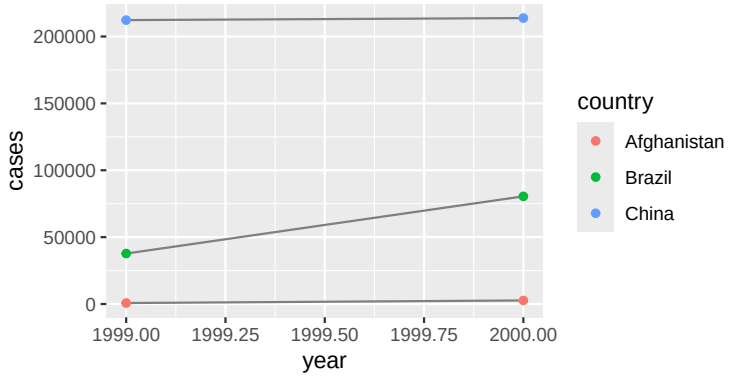
```
## # A tibble: 6 x 5
##   country      year  cases population  rate
```

```
##   <chr>       <dbl>  <dbl>      <dbl> <dbl>
## 1 Afghanistan  1999    745   19987071 0.373
## 2 Afghanistan  2000   2666   20595360 1.29
## 3 Brazil       1999  37737  172006362 2.19
## 4 Brazil       2000  80488  174504898 4.61
## 5 China        1999 212258 1272915272 1.67
## # i 1 more row
```

```
table1 %>% count(year, wt = cases)                    # cases per year
```

```
## # A tibble: 2 x 2
##    year      n
##   <dbl>  <dbl>
## 1  1999 250740
## 2  2000 296920
```

```
ggplot(table1, aes(year, cases)) +
  geom_line(aes(group = country), colour = "grey50") +
  geom_point(aes(colour = country))
```



5.4 Gathering and spreading

Motivation. Most real data are untidy in one of two ways: one variable is spread over several columns, or one observation is spread over several rows. Gathering and spreading are the two inverse operations that fix these.

Gathering. In `table4a` the column names `1999`, `2000` are values of the variable `year`, and each row holds two observations. `gather()` collects these columns into a pair of variables: the **key** (the variable whose values were column names) and the **value** (the variable whose values were in the cells).

```
table4a
```

```
## # A tibble: 3 x 3
##   country     `1999` `2000`
##   <chr>        <dbl>  <dbl>
## 1 Afghanistan    745   2666
## 2 Brazil       37737  80488
## 3 China       212258 213766
```

```
tidy4a <- table4a %>% gather(`1999`, `2000`, key = "year", value = "cases")
tidy4b <- table4b %>% gather(`1999`, `2000`, key = "year", value = "population")
left_join(tidy4a, tidy4b)
```

```
## # A tibble: 6 x 4
##   country     year   cases population
##   <chr>       <chr>  <dbl>      <dbl>
## 1 Afghanistan 1999     745   19987071
```

```
## 2 Brazil      1999   37737  172006362
## 3 China       1999  212258 1272915272
## 4 Afghanistan 2000    2666   20595360
## 5 Brazil      2000   80488  174504898
## # i 1 more row
```

The two tidied tables are combined by `left_join()` (Lecture 6). Gathering makes tables longer.

Spreading is the inverse. In `table2` an observation (country, year) is spread over two rows; `spread()` turns the values of the key column `type` into column names, filled from `count`. Spreading makes tables wider.

```
spread(table2, key = type, value = count)
```

```
## # A tibble: 6 x 4
##   country      year  cases population
##   <chr>       <dbl>  <dbl>      <dbl>
## 1 Afghanistan  1999    745   19987071
## 2 Afghanistan  2000   2666   20595360
## 3 Brazil       1999  37737  172006362
## 4 Brazil       2000  80488  174504898
## 5 China        1999 212258 1272915272
## # i 1 more row
```

pivot_longer() and pivot_wider() [Supplement]

In current tidyr, `gather()` and `spread()` are superseded by `pivot_longer()` and `pivot_wider()`, which do the same with clearer argument names; the old functions still work. The calls above become

```
table4a %>% pivot_longer(c(`1999`, `2000`), names_to = "year", values_to = "cases")
table2 %>% pivot_wider(names_from = type, values_from = count)
```

`pivot_longer()` also has `names_transform` and `values_drop_na` arguments, so the type of `year` and missing cells can be handled in the same call.

5.5 Separating and uniting

`table3` stores two variables, cases and population, in one column `rate`. `separate()` splits a column at a separator (by default any non-alphanumeric character):

```
table3 %>% separate(rate, into = c("cases", "population"), convert = TRUE)
```

```
## # A tibble: 6 x 4
##   country      year  cases population
##   <chr>       <dbl>  <int>      <int>
## 1 Afghanistan  1999    745   19987071
## 2 Afghanistan  2000   2666   20595360
## 3 Brazil       1999  37737  172006362
## 4 Brazil       2000  80488  174504898
## 5 China        1999 212258 1272915272
## # i 1 more row
```

Without `convert = TRUE` the new columns stay character, since they were split from a string. If `sep` is a number, it is a position: `separate(year, into = c("century", "year"), sep = 2)` splits after the second character.

`unite()` is the inverse: `table5` has the year split into century and year, and

```
table5 %>% unite(new, century, year, sep = "")
```

```
## # A tibble: 6 x 3
##   country     new   rate
```

```
##   <chr>       <chr> <chr>
## 1 Afghanistan 1999  745/19987071
## 2 Afghanistan 2000  2666/20595360
## 3 Brazil      1999  37737/172006362
## 4 Brazil      2000  80488/174504898
## 5 China       1999  212258/1272915272
## # i 1 more row
```

pastes them back together (the default separator is _).